

MODULE 5.1 · Concept 1 of 7

ReAct Framework: Reasoning + Acting

Interleaving Chain-of-Thought with tool calls — the dominant paradigm for production agents

10 min

Duration

B3 – Apply

Bloom's

L3 – Advanced

Level

TH-000006

Prerequisite

AI-AG-IN-TH-000001

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — WHY

The Problem with Reasoning-Only and Acting-Only

Neither approach alone is sufficient — ReAct combines both

Reasoning-Only: Chain-of-Thought Without Tools

The model generates plausible but ungrounded answers

User: "What is the current Nifty 50 index value?"

CoT-Only LLM: "Let me think... the Nifty 50 has been trending around 22,000–23,000 in recent months. Based on market conditions, I estimate it's approximately 22,450 today."

✗ This is a hallucination. The model has no access to real-time data. The actual value could be anything.

Why it fails:

- Chain-of-Thought reasons correctly about HOW to solve the problem
- But has NO access to current data — knowledge frozen at training time
- Generates confident-sounding answers from stale training data
- Cannot verify its own claims — no feedback from the real world
- Result: plausible but wrong. The worst kind of error — convincing hallucinations.

Acting-Only: Tools Without Reasoning

The agent calls tools but nobody can understand why

User: "Compare revenue of TCS and Infosys for 2024"

Act-Only Agent:

Action: search("TCS") → Result: company overview page

Action: search("Infosys revenue") → Result: careers page

Action: calculator("0 - 0") → Result: 0

Answer: "The difference is 0."

✗ No reasoning about WHAT to search, HOW to extract data, or WHETHER results are valid.

Why it fails:

- Tools are called without planning — random or pattern-matched tool selection
- No interpretation of results — agent doesn't check if search returned relevant data
- No transparent decision process — impossible to debug why it went wrong
- No error recovery — if first search fails, agent doesn't reformulate the query
- Result: opaque failures. You can see WHAT happened but not WHY.

The Gap: Neither Approach Alone Is Sufficient

Reasoning needs grounding. Acting needs transparency.

	Reasoning-Only (CoT)	Acting-Only (Tools)	What's Missing
<u>Grounding</u>	✗ No real data	✓ Has real data	CoT needs tool access
<u>Transparency</u>	✓ Shows thinking	✗ Opaque decisions	Tools need reasoning trace
<u>Accuracy</u>	Low (hallucination)	Low (random calls)	Both need the other
<u>Debuggability</u>	✓ Can read logic	✗ Can't explain why	Tools need visible decisions
<u>Self-correction</u>	✗ No feedback loop	✗ No error detection	Both need observation → reason

The solution: Interleave reasoning traces WITH action steps in the same context window. This is ReAct.

critic

Output Phasing
(200+ tools)

Step 1 → Call tools
Step 2
Step N → " "

SECTION 2 — WHAT

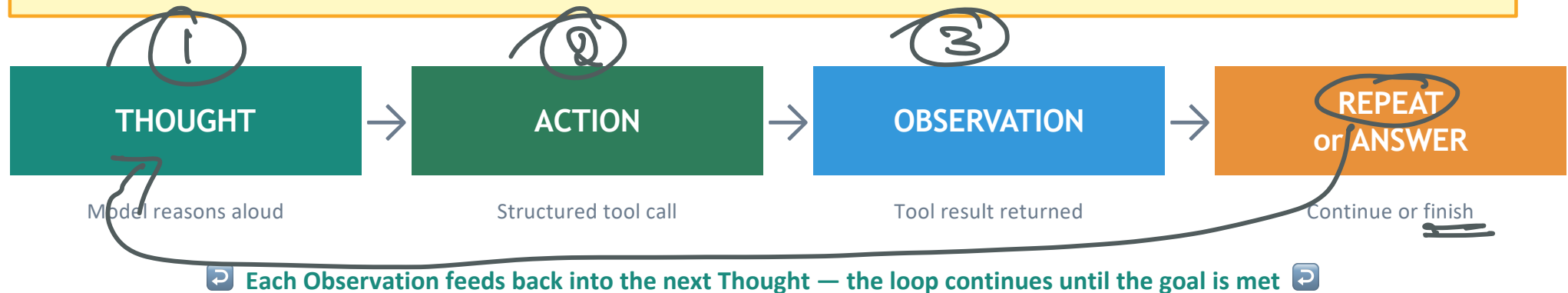
The ReAct Framework

Yao et al. (2022) — Synergizing Reasoning and Acting in Language Models

ReAct: Interleave Thought + Action + Observation

A prompting framework where reasoning and acting happen together in one context

ReAct (Yao et al., 2022) is an agent framework that interleaves **reasoning traces (Chain-of-Thought)** with **action steps (tool calls)** in a single context window. The model alternates between thinking and acting, making its decisions transparent and correctable.



Key insight: Thoughts make actions intentional. Actions make thoughts grounded. Together they produce reliable, debuggable agent behavior.

The Original Paper: Yao et al. (ICLR 2023)

Benchmark results that proved ReAct works

Key Findings from the Paper:

HotpotQA (multi-hop QA): ReAct outperformed CoT-only by 6% accuracy. Eliminated hallucination on factual lookups.

FEVER (fact verification): ReAct achieved 60.9% vs CoT-only 56.3%. Tool grounding reduced false claims.

ALFWorld (interactive tasks): ReAct solved 71% of tasks vs Act-only 45%. Reasoning before acting was critical.

WebShop (web navigation): ReAct achieved 40% success vs Act-only 29%. Planning improved tool sequence quality.

Impact: ReAct became the default architecture for production agents. LangChain's `create_react_agent`, OpenAI's Assistants API, and Claude's tool use all implement the ReAct pattern. The paper has 3,000+ citations in 3 years.

Why Interleaving Works

Reasoning grounds actions. Actions ground reasoning. Both are visible.

Reasoning → Grounds Actions

Before calling a tool, the model explains WHY:
'I need the current price, so I should search for it'
This prevents random tool calls and makes the agent's intent visible for debugging.

Actions → Ground Reasoning

After calling a tool, the result enters the context:
'Search returned: Nifty 50 = 22,474'
Now reasoning is based on REAL data, not hallucinated estimates.

Both Are Visible → Debuggable

Every Thought, Action, and Observation is logged in the context window. When something goes wrong, you can read the trace backwards to find exactly where the agent made a bad decision. This is impossible with opaque acting-only agents.

ReAct Prompt Template Anatomy

The system prompt that makes agents follow the Thought→Action→Observation pattern

System Prompt Template:

1 Answer the user's question using the available tools.
You must follow this EXACT format for every step:

Thought: [reason about what you need to do next]

Action: [tool_name(param1="value", param2="value")]

Observation: [result returned by the tool] ~~DO NOT GENERATE THIS~~

Repeat Thought/Action/Observation as needed.

When you have enough information, respond with:

Thought: I now have enough information to answer.

Answer: [your final response to the user]

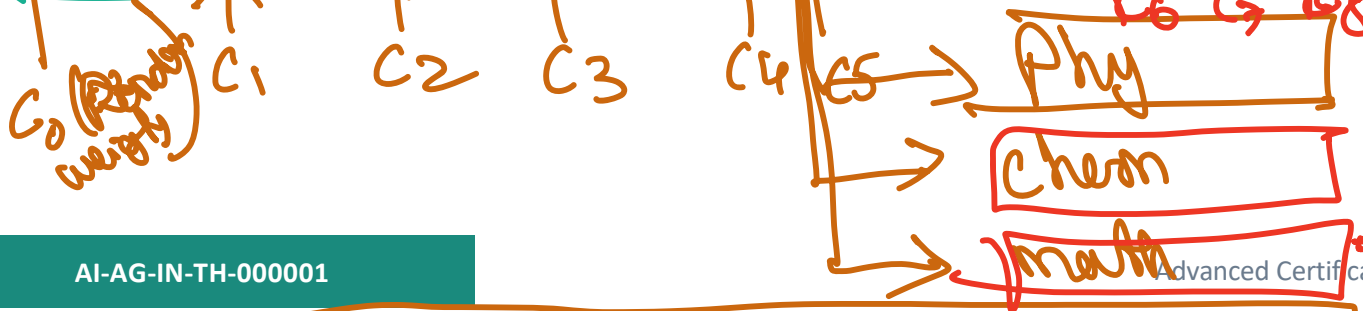
SLM [domain-specific]
- web-search
- calc
- exec (code)
LLM 1 [thinking mode]
LLM 2 [SLM, smaller model]

LLM training lifecycle

S1: Pre-training (CPT)

S2: mid training

S3: SFT | S4: I.T | S5: ALM



Handwritten notes in red boxes: 'Theo | Prof | Anshu', 'EMCP', and 'Numerical Sol'.

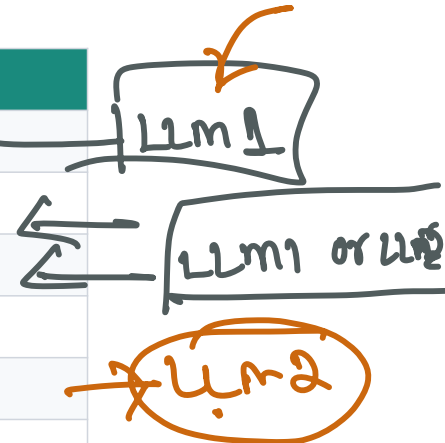
AI-AG-IN-TH-000001

Advanced Certification in Agentic & Generative AI | IISc Bangalore

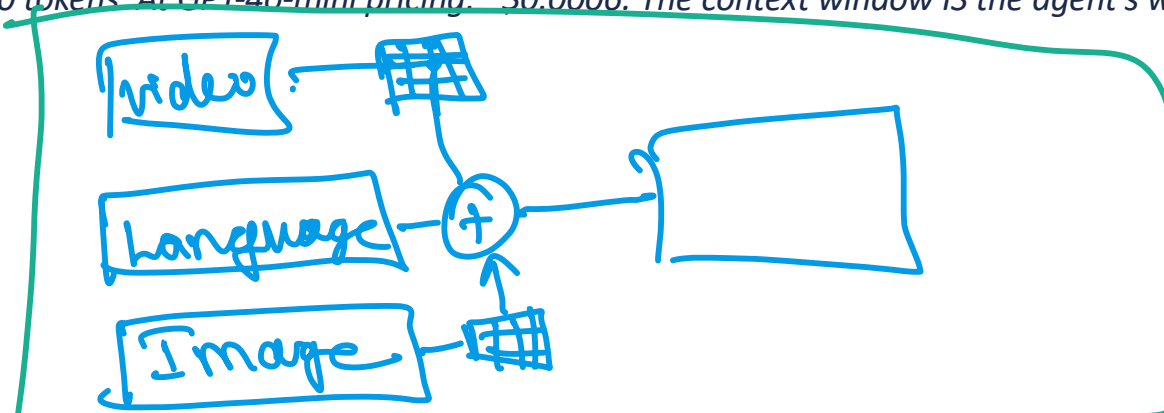
How ReAct Grows in the Context Window

Each iteration appends ~80–150 tokens of Thought + Action + Observation

Iteration	Content Added	Tokens	Cumulative
Start	System prompt + User query	~500	500
Iter 1	Thought 1 + Action 1 (tool call JSON)	~80	580
	Observation 1 (tool result injected)	~100	680
Iter 2	Thought 2 + Action 2	~80	760
	Observation 2	~60	820
Iter 3	Thought 3 (final) + Answer	~100	920



A typical 3-iteration ReAct trace uses ~920 tokens. At GPT-4o-mini pricing: ~\$0.0006. The context window IS the agent's working memory.



Three-Way Comparison: ReAct vs. CoT vs. Act-Only

Why ReAct wins on the dimensions that matter

Dimension	CoT-Only	Act-Only	ReAct
Accuracy	Low (hallucination)	Low (random calls)	High (grounded reasoning)
Transparency	High (visible thinking)	Low (opaque)	High (Thought + Action logged)
Grounding	None (no tool access)	Yes (has tools)	Yes (reasoning + tools)
Debuggability	Read reasoning trace	Read tool log only	Read both + see decisions
Cost per query	Low (~200 tok)	Medium (~500 tok)	Medium (~900 tok)
Self-correction	Cannot (no feedback)	Cannot (no reasoning)	Can (observe → rethink)

SECTION 3 — WHERE

Production Adoption and Failure Modes

ReAct is the default — but it's not perfect

ReAct as the Default Production Architecture

Every major agent framework implements this pattern

Framework / Product	How It Uses ReAct	Since
LangChain <code>create_react_agent</code>	Explicit Thought→Action→Observation loop with tool executor	2023
OpenAI Assistants API	Tool calls interleaved with assistant reasoning messages	2023
Claude <code>tool_use</code>	Thinking blocks + <code>tool_use</code> blocks in same response	2024
LangGraph	ReAct as a state-machine node type within agent graphs	2024
AutoGen (Microsoft)	ReAct embedded in multi-agent conversation patterns	2023
CrewAI	Role-based agents each running internal ReAct loops	2024

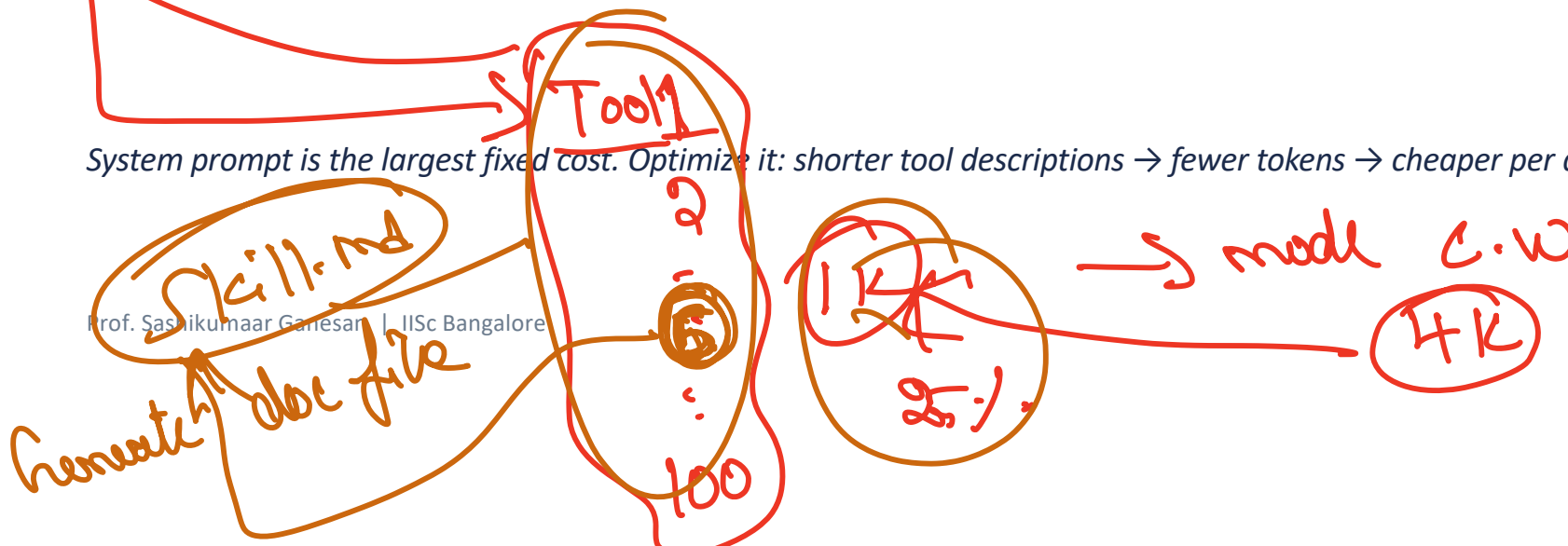
If you build an agent today with any major framework, you are using ReAct — whether you know it or not.

Token Cost Anatomy of a ReAct Trace

Where the tokens go — and what each step costs

Component	Tokens	Cost (GPT-4o-mini)	% of Total
System prompt	~400	\$0.00006	43%
Thought (per iter)	~40	\$0.000024	4%
Action JSON (per iter)	~40	\$0.000024	4%
Observation (per iter)	~80	\$0.000012	9%
Final answer	~100	\$0.00006	11%
Total (3 iters)	~920	\$0.0006	100%

System prompt is the largest fixed cost. Optimize it: shorter tool descriptions → fewer tokens → cheaper per query.



When ReAct Fails: Three Common Failure Modes

Knowing the failure patterns helps you design better agents

1. Hallucinated Tool Call

Agent calls `search_flights()` but only `search()` and `calc()` exist. The tool name was invented by the model.

Fix: Validate tool names against registry before execution. Return clear error message.

2. Infinite Loop

Agent searches → gets result → decides it needs more info → searches again → repeat. 20+ iterations without converging.

Fix: Set `max_iterations` limit. Add convergence check: 'if same info retrieved twice, stop'.

3. Wrong Tool Selection

Agent uses `calculator()` to look up a stock price instead of `search()`. Tool description was ambiguous.

Fix: Write precise tool descriptions. Include negative examples: 'Do NOT use for lookups'.

All three failures are detectable in the trace. That's the advantage of ReAct — failures are visible.

Key Takeaways

The essential points from this concept

ReAct interleaves reasoning + acting in one context window — the dominant agent architecture.

Reasoning-only hallucinates; acting-only is opaque. ReAct solves both by combining them.

Yao et al. (2022) proved it: 6% accuracy gain on HotpotQA, 26% gain on ALFWorld.

Every major framework uses ReAct: LangChain, OpenAI Assistants, Claude, LangGraph.

Three failure modes to watch: hallucinated tools, infinite loops, wrong tool selection.

Next: AI-AG-IN-TH-000002 — Thought-Action-Observation Loop (deep dive into the four steps)

AI-AG-IN-TH-000001 Complete

ReAct Framework: Reasoning + Acting

Concept 1 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi